

Title: Ternary Oscillating Logic for Binary Systems
Inventor: Jonathan M Schack
Filing Type: Utility Patent Application (Small Entity)

[0001] FIELD OF THE INVENTION:

The invention relates to artificial intelligence and computational logic systems. More specifically, it concerns methods and systems for detecting, sustaining, and classifying undecidable reasoning states using a ternary (T/F/N-U) oscillating logic kernel implemented within binary computing architectures.

[0002] BACKGROUND OF THE INVENTION:

Binary reasoning systems, including large language models, symbolic engines, and rule-based inference systems, frequently encounter paradoxes, incomplete axioms, or ambiguous propositions. Conventional systems collapse such inputs into binary outputs (TRUE/FALSE) or probabilistic guesses, producing hallucinations, contradictions, or unstable reasoning chains. Existing approaches rely on token prediction, reinforcement learning, or confidence-based heuristics, none of which maintain undecidability as a stable computational state. No current system provides a structured oscillatory mechanism for sustaining, measuring, and classifying undecidable propositions.

Conventional binary architectures waste significant computational resources (CPU/GPU cycles and energy) processing invalid reasoning chains that inevitably lead to hallucination. There is a need for a computational gating mechanism that halts wasteful processing at the logic kernel level, thereby improving the specific power efficiency and throughput of AI hardware.

Operating in a ternary space (TRUE, FALSE, NEUTRAL, UNDECIDABLE) sustains oscillation until collapse conditions are met, classifies non-collapse states for downstream reasoning, and improves reliability and reduces hallucination rates in AI systems.

Additionally, existing AI systems deployed in high-stakes domains (medical diagnosis, autonomous systems, legal analysis) lack formal mechanisms to refuse output when evidence is insufficient, leading to increased liability and reduced safety. The present invention addresses this gap by providing a mathematically verifiable undecidability state.

Unlike conventional binary or fuzzy logic systems, the present invention introduces a dedicated oscillation kernel that treats UNDECIDABLE as a first-class computational state implemented via complex-plane dynamics and volatility-based collapse criteria. Existing systems do not sustain undecidability as a stable state, nor do they classify non-collapse outcomes into `ontological_error`, `paradox`, `epistemic_gap`, and `structural_noise` for downstream reasoning.

[0003] SUMMARY OF THE INVENTION:

The invention provides a ternary oscillating logic kernel that operates on a complex-valued state vector. Contradiction density, evidence score, and domain indicators are injected into the kernel. Iterative gain, loss, and entropy functions produce oscillatory behavior whose volatility determines whether the system collapses to TRUE, FALSE, NEUTRAL, or remains UNDECIDABLE.

In benchmark testing on Sudoku puzzles (SER 11.8-12.8), the oscillating logic kernel detected computational futility (volatility > 0.7) within 60 cycles, triggering handoff to symbolic solver,

reducing token usage by 93% and solving time by 85% compared to pure autoregressive generation.

Non-collapse states are classified into categories including `ontological_error`, `paradox`, `epistemic_gap`, and `structural_noise`. The system thereby prevents premature binary collapse, reduces hallucinations, and enables adaptive escalation to specialized reasoning modules. By introducing a ternary oscillating logic kernel implemented on binary computing hardware, the invention transforms general-purpose computing systems into safety-critical reasoning engines by: (1) reducing invalid reasoning branches by 60-90% (as demonstrated in exact-cover problem benchmarks), (2) decreasing token generation in generative models by halting computation when evidence is insufficient, thereby improving computational efficiency and reducing energy consumption, and (3) enabling real-time classification of failure modes for adaptive routing, which was previously impossible in binary systems.

This kernel acts as a hardware-agnostic throttle that physically prevents the processor from executing instruction cycles on ungrounded data.

[0004] EVOLUTION NOTE:

This invention evolved from the Chaos AI-OS framework, originally published as "Chaos Persona" (CRB) on June 3, 2025 (Zenodo DOI: 10.5281/zenodo.1579582), moved to <https://cai-os.com> and <https://github.com/elxaber/chaos-persona/>. Earlier embodiments focused on entropy-driven hallucination mitigation. The present invention introduces a dedicated oscillation kernel for ternary logic, enabling sustained undecidability and structured classification.

[0005] ABSTRACT:

A system and method for ternary oscillating logic in binary computing systems. A complex-valued state vector is initialized using contradiction density and evidence scores derived from query analysis and is iteratively updated through gain, loss, entropy, and decay functions to produce oscillatory behavior. Volatility of the complex state is monitored to determine collapse to TRUE, FALSE, or NEUTRAL, or persistence as an UNDECIDABLE state. Persistent non-collapse states are classified into categories including `ontological_error`, `paradox`, `epistemic_gap`, and `structural_noise`. The ternary oscillating logic kernel functions as a computational gate that prevents premature binary collapse, reduces hallucinations by refusing output when evidence is insufficient, and improves computational efficiency by inhibiting downstream processing on unstable or paradoxical reasoning chains.

[0006] DETAILED DESCRIPTION OF THE INVENTION:

Initialization and State Vector Allocation:

A complex state vector $\$z\$$ is initialized and allocated to a dedicated memory register. Contradiction density (a real-valued metric between 0.0 and 1.0 representing the magnitude of semantic opposition between extracted propositions) and evidence score (0.0 to 1.0) are computed via token-level analysis of the input query. These values are mapped to the initial control parameters of the oscillation kernel.

Computational State-Gating Mechanism:

Unlike conventional logic systems that process all inputs to a binary conclusion, the present invention functions as a volatility-based state gate. The kernel monitors the Volatility of the state vector (defined as the standard deviation of the vector's magnitude over a moving temporal window stored in a history buffer).

This volatility metric acts as a digital circuit breaker:

High Volatility (Open Circuit):

When volatility exceeds a pre-defined threshold (e.g., 0.04), the system inhibits the transmission of the state vector to downstream generation layers. This prevents the processor from expending computational cycles on invalid or paradoxical reasoning chains.

Low Volatility (Closed Circuit):

When volatility settles below the threshold, the gate opens, allowing the resolved state (TRUE/FALSE) or the classified exception (e.g., `ontological_error`) to propagate to the output buffer.

Domain Extraction:

Simultaneously, a classifier identifies the query domain (e.g., math, physics, logic) by comparing input tokens against a stored hash map of domain-specific keywords. Unknown domains trigger an `epistemic_gap` flag, which modifies the collapse threshold parameters dynamically.

Neutral State Definition:

In one embodiment, the system distinguishes between a NEUTRAL state and an UNDECIDABLE state. A NEUTRAL state is a collapsed outcome produced when the oscillation stabilizes below the volatility threshold, but the real component of the complex state z remains within a bounded region around zero (e.g., $-0.5 \leq \text{Re}(z) \leq 0.5$). NEUTRAL therefore represents a resolved state indicating insufficient directional evidence toward TRUE or FALSE, but not a failure of the oscillation to collapse. By contrast, an UNDECIDABLE state occurs when the oscillation fails to converge within the allotted cycle count or when volatility remains above the collapse threshold, indicating persistent instability. UNDECIDABLE is therefore a non-collapsed state that triggers classification and escalation, whereas NEUTRAL is a collapsed state that may be safely propagated to downstream systems.

Bounded Oscillation and Cycle Limits:

In one embodiment, the oscillation performed by the ternary logic kernel is explicitly bounded to prevent unbounded looping or uncontrolled compute expenditure. Let T_{max} denote the maximum permitted number of oscillation cycles. The kernel iteratively updates the complex state z_t for $t = 0, 1, 2, \dots, T_{\text{max}}$, after which oscillation is forcibly terminated if collapse has not occurred. The volatility functional $V(z_{t-k+1}, \dots, z_t)$ is evaluated at each cycle, and collapse is permitted only when V falls below a predetermined threshold. If V remains above the threshold for all $t \leq T_{\text{max}}$, the system declares an UNDECIDABLE state and halts further oscillation. This bounded-iteration design ensures that the kernel cannot enter an infinite loop and that computational cost is strictly limited to $O(T_{\text{max}})$ per query. As a result, the oscillation kernel functions as a deterministic compute throttle: it prevents downstream inference modules

from consuming processor cycles on unstable or paradoxical reasoning chains, thereby reducing energy usage and improving overall system efficiency.

Extended-Cycle and Soft-Set Parameters:

In one embodiment, the maximum oscillation cycle count T_{max} and the history window size k are soft-set parameters that may be adjusted based on application requirements. For example, a default configuration may employ $T_{\text{max}} = 60$ cycles with a history window of $k = 5$ entries to ensure rapid convergence and minimal compute expenditure. In alternative embodiments, T_{max} may be increased (e.g., 200–300 cycles) to permit extended semantic stabilization, deeper paradox exploration, or controlled “semantic heat death” analysis in research or diagnostic modes. These extended-cycle configurations remain bounded and deterministic, as T_{max} is always explicitly specified by the system or user. In another embodiment, T_{max} may be set to an application-defined value that effectively permits indefinite oscillation, provided that the oscillation remains under supervisory control and is used for specialized tasks such as anomaly detection, adversarial probing, or epistemic uncertainty mapping. In all cases, the oscillation kernel remains governed by explicit cycle limits, ensuring that no unbounded or uncontrolled looping occurs unless intentionally configured for a specific analytical purpose.

Oscillation Cycle (Primary Embodiment):

In one embodiment, the ternary oscillating logic kernel performs an iterative oscillation cycle on a complex-valued state z .

Let z be a complex number, where the real part represents directional tendency toward TRUE or FALSE and the imaginary part represents accumulated uncertainty. Let gain be a positive real-valued parameter in the range $(0, 1)$, and let contradiction_density be a real-valued parameter between 0.0 and 1.0. Let decay be a real-valued parameter between 0.0 and 1.0 controlling the attenuation of the state magnitude.

The kernel functions as a cryptographic-style lock or state-gate. Just as a checksum prevents corrupt data validation, the volatility threshold mechanically prevents the state vector from passing to the output buffer until 'settling' occurs. This ensures that the downstream transformer or inference engine does not expend distinct compute cycles on invalid states.

The oscillation cycle consists of the following functions:

1. Truth-Seer function:

The Truth-Seer function moves the real part of z toward +1.0, representing attraction toward a TRUE resolution when the system pushes in the direction of truth-seeking.

Given the current state z , the Truth-Seer update is defined as:

$$z_{\text{ts}} = z + \text{gain} * (1.0 - \text{Re}(z))$$

where $\text{Re}(z)$ denotes the real part of z . The imaginary part of z_{ts} is unchanged in this embodiment.

2. Lie-Weaver function:

The Lie-Weaver function moves the real part of z toward -1.0, representing attraction toward a FALSE resolution when the system pushes in the direction of contradiction or falsification.

Given the output of the Truth-Seer function z_{ts} , the Lie-Weaver update is defined as:

$$z_{lw} = z_{ts} - \text{gain} * (1.0 + \text{Re}(z_{ts}))$$

The imaginary part of z_{lw} is unchanged in this embodiment.

3. Entropy-Knower function:

The Entropy-Knower function modulates both magnitude and phase of the complex state as a function of `contradiction_density`. High `contradiction_density` produces stronger rotational effects, while low `contradiction_density` produces weaker rotation and more linear behavior.

Let `rotation_strength` be defined as:

$$\text{rotation_strength} = (\text{contradiction_density})^2$$

The Entropy-Knower function computes a complex phase factor as a convex combination of a real component and an imaginary component:

$$\text{phase_factor} = (1.0 - \text{rotation_strength}) + \text{rotation_strength} * i$$

where i is the imaginary unit ($\sqrt{-1}$).

The Entropy-Knower update multiplies the current state by the phase factor:

$$z_{ek} = z_{lw} * \text{phase_factor}$$

This operation rotates and scales z_{lw} in the complex plane in proportion to the square of the `contradiction_density`.

4. Decay function:

The Decay function gradually reduces the magnitude of the complex state to avoid unbounded growth and to stabilize the oscillation over time.

Given the output of the Entropy-Knower function z_{ek} , the Decay function is defined as:

$$z_{next} = z_{ek} * \text{decay}$$

where `decay` is a real number between 0.0 and 1.0. A decay value close to 1.0 preserves magnitude, while a value closer to 0.0 produces rapid attenuation.

In each oscillation cycle, the kernel updates z as:

$$z := z_{next}$$

and appends z_{next} to a bounded history buffer. Volatility is measured over the magnitudes of recent history entries, and collapse or non-collapse classification is determined based on volatility thresholds as described elsewhere in this specification.

Volatility Measurement:

Volatility is computed as the standard deviation of magnitudes in a sliding history window.

Collapse Logic:

If `volatility < threshold` and history is full:

`real(z) > 0.5` → TRUE

`real(z) < -0.5` → FALSE

otherwise → NEUTRAL (unless contradiction density is high)

Non-Collapse Classification:

If oscillation persists:

`evidence=0` and axiom absent → `ontological_error`

`contradiction_density > 0.85` → `paradox`

new domain and low density → epistemic_gap
else → structural_noise

[0007] Alternative Oscillation Embodiments:

Alternative Embodiment A: Phase-Based Truth-Seer and Lie-Weaver:

In another embodiment, the Truth-Seer and Lie-Weaver functions operate directly in polar form using the magnitude and phase of the complex state instead of only the real part.

Let z have magnitude r and phase θ (theta), such that:

$$z = r * \exp(i * \theta)$$

The Truth-Seer function rotates the phase toward a "truth" direction and increases magnitude toward an upper bound:

$$\theta_{ts} = \theta + \text{gain} * (1.0 - \text{contradiction_density})$$

$$r_{ts} = r + \text{gain} * (1.0 - r)$$

$$z_{ts} = r_{ts} * \exp(i * \theta_{ts})$$

The Lie-Weaver function rotates the phase toward a "false" direction and decreases magnitude:

$$\theta_{lw} = \theta - \text{gain} * \text{contradiction_density}$$

$$r_{lw} = r_{ts} - \text{gain} * r_{ts}$$

$$z_{lw} = r_{lw} * \exp(i * \theta_{lw})$$

The Entropy-Knower function then adds a sinusoidal perturbation to the phase as a function of contradiction_density:

$$\theta_{ek} = \theta_{lw} + \text{gain} * \sin(2 * \pi * \text{contradiction_density})$$

$$z_{ek} = r_{lw} * \exp(i * \theta_{ek})$$

The Decay function applies:

$$z_{next} = z_{ek} * \text{decay}$$

Alternative Embodiment B: Logistic Map-Based Entropy-Knower:

In another embodiment, the Entropy-Knower function uses a chaotic logistic map to modulate the magnitude of z .

Let $r_{logistic}$ be a control parameter in the range (3.5, 4.0), chosen based on contradiction_density, and let x be a real-valued logistic state derived from the magnitude of z :

$$x = \min(1.0, \max(0.0, |z|))$$

The Entropy-Knower update computes:

$$x_{next} = r_{logistic} * x * (1.0 - x)$$

and sets the new magnitude of z as:

$$r_{next} = x_{next}$$

The phase of z is optionally rotated based on contradiction_density, for example:

$$\theta_{next} = \theta + \text{gain} * \text{contradiction_density}$$

The updated complex state is:

$$z_{ek} = r_{next} * \exp(i * \theta_{next})$$

followed by the Decay function:

$$z_{next} = z_{ek} * \text{decay}$$

Alternative Embodiment C: Nonlinear Activation-Based Truth-Seer

In another embodiment, the Truth-Seer function uses a nonlinear activation such as hyperbolic tangent to push the real part toward saturation:

$$z_{ts_real} = \tanh(\text{Re}(z) + \text{gain} * (1.0 - \text{Re}(z)))$$

$$z_{ts_imag} = \text{Im}(z)$$

$$z_{ts} = z_{ts_real} + i * z_{ts_imag}$$

The Lie-Weaver and Entropy-Knowler functions can be defined analogously, and the Decay function is applied as described in other embodiments.

In another embodiment, the ternary oscillating logic kernel is implemented in hardware using digital logic circuits, FPGA fabric, or ASIC components configured to perform complex-number operations, oscillation updates, and volatility measurement.

In a hardware embodiment, the complex state z is stored in dual floating-point registers. The 'Truth-Seer' and 'Lie-Weaver' functions are implemented via arithmetic logic units (ALUs) performing single-cycle multiply-accumulate operations. The 'Volatility Check' is a comparator circuit against a stored reference voltage or digital threshold.

These alternative embodiments demonstrate that the ternary oscillating logic kernel is not limited to a single closed-form equation. Any combination of gain, loss, entropy, and decay operations that produces a sustained oscillation in the complex plane, monitored for volatility and collapse, falls within the scope of the invention.

[0008] SYSTEM ARCHITECTURE:

Input → Domain Extraction → Evidence Scoring → Oscillation Loop → Volatility Check → Collapse or Classification.

Flow of operation

Receive query → Extract contradiction density → Extract evidence score → Extract domain → Initialize complex state → Run oscillation loop → Measure volatility → Collapse or classify → Output result.

[0009] Technical Advantages:

The ternary oscillating logic kernel described herein provides several technical advantages over prior systems. First, it implements a stable UNDECIDABLE state in a binary computing environment without requiring ternary hardware. Second, it uses volatility in a complex state space as a measurable criterion for logical collapse, thereby reducing premature and unsupported resolutions. Third, it generates explicit non-collapse classifications that can be

logged, audited, and consumed by other components, improving transparency and controllability of artificial intelligence reasoning.

[0010] DIFFERENTIATION FROM PRIOR ART:

Kleene's Three-Valued Logic (1938):

Kleene introduced UNKNOWN as a third truth value for database nulls. However, Kleene logic does not provide an oscillatory mechanism, volatility measurement, or automatic classification of undecidable states. The present invention differs by sustaining undecidability through complex-plane oscillation and measuring volatility to determine collapse.

Fuzzy Logic (Zadeh, 1965):

Fuzzy logic uses continuous [0,1] membership values. However, fuzzy logic does not distinguish between types of uncertainty (paradox vs. missing evidence vs. ambiguity), nor does it provide a formal collapse criterion. The present invention differs by classifying non-collapse states into `ontological_error`, `paradox`, `epistemic_gap`, and `structural_noise`, enabling targeted remediation.

Constitutional AI (Anthropic, 2022):

Constitutional AI uses reinforcement learning from human feedback to align model outputs with safety principles. However, Constitutional AI does not implement a ternary state space, relies on soft probabilistic constraints rather than hard thresholds, and lacks formal mechanisms to sustain undecidability. The present invention differs by implementing immutable collapse thresholds and maintaining UNDECIDABLE as a stable computational state.

None of the prior systems combine oscillatory dynamics in a complex state space with volatility-based collapse criteria and structured classification of non-collapse states.

[0011] EXAMPLES OF INDUSTRIAL APPLICATION:

Medical Diagnosis Systems:

An AI system analyzing patient symptoms with insufficient data (no imaging, labs, or physical exam) triggers UNDECIDABLE + `ontological_error`, refusing diagnosis and requiring additional data collection, thereby reducing misdiagnosis rates and malpractice liability.

Autonomous Weapons Targeting:

A targeting system encountering ambiguous visual data (potential civilian vs. combatant) enters sustained oscillation, maintaining UNDECIDABLE status and requiring human override, thereby complying with International Humanitarian Law requirements for meaningful human control.

Legal Case Analysis:

An AI reviewing conflicting circuit court precedents detects high contradiction density, classifies it as a paradox, and escalates to human attorney review rather than hallucinating a false synthesis.

Exact-Cover Problem Solving (Computational Efficiency Demonstration) In benchmark testing on Sudoku puzzles rated SER 11.8-12.8 (among the hardest known 17-clue puzzles), the ternary oscillating logic kernel detected computational futility within 60 oscillation cycles (average wall-clock time: 4.1 seconds). Upon detecting sustained oscillation (volatility > 0.7), the

system triggered handoff to a symbolic Dancing Links solver, completing the remaining cells deterministically.

Performance comparison (M3-Max hardware, identical temperature settings): - Pure autoregressive generation (GPT-4, Gemini, Grok): * Average time: 83 seconds * Token usage: 40,000+ * Wrong guesses: 50+ * Success rate: 12% (many puzzles declared "unsolvable"). With CPOL pre-processing: * Average time: 4.9 seconds (17x faster) * Token usage: 2,500 (93% reduction) * Wrong guesses: 0 * Success rate: 100% (9 consecutive perfect solves) This demonstrates the kernel's ability to detect combinatorial explosion early, preventing wasteful computation and improving both accuracy and energy efficiency.

[0012] FORMAL SYSTEM ARCHITECTURE:

In one embodiment, the ternary oscillating logic system can be expressed as a composition of functions operating on an input query q .

Let q be an input query drawn from a space Q of text sequences. The system applies the following transformations:

1. Semantic analysis and signal extraction:

A contradiction density extractor $C: Q \rightarrow [0, 1]$ maps the query q to a real-valued contradiction density:

$$c = C(q)$$

An evidence scorer $E: Q \rightarrow [0, 1]$ maps the query q to a real-valued evidence score:

$$e = E(q)$$

A domain classifier $D: Q \rightarrow \Sigma$ maps the query q to a domain label in a finite set Σ of domains:

$$d = D(q)$$

2. State initialization:

A state initializer $S: [0, 1] \times [0, 1] \rightarrow \mathbb{C}$ maps the contradiction density and evidence score to an initial complex state z_0 :

$$z_0 = S(c, e)$$

In one embodiment, S sets $z_0 = \text{confidence} + 0i$, where the confidence is a function of c and e , and i is the imaginary unit.

3. Oscillation kernel:

The oscillation kernel K operates on the complex state z and the contradiction density c to produce a sequence $\{z_t\}$ in $\mathbb{C}$:

$$z_{t+1} = K(z_t, c)$$

for $t = 0, 1, 2, \dots, T$, where T is the maximum cycle count. The kernel K is defined as the composition of gain, loss, entropy, and decay operations:

$$K(z, c) = \text{Decay}(\text{Entropy}(\text{LieWeaver}(\text{TruthSeer}(z)), c))$$

In other embodiments, alternative formulations of K , including polar-coordinate updates and logistic-map-based updates, are used as described in this specification.

4. Volatility measurement:

A volatility functional $V: \mathbb{C}^k \rightarrow \mathbb{R}_{\geq 0}$ maps a finite history window of complex states to a non-negative real-valued volatility:

$$v = V(z_{t-k+1}, \dots, z_t)$$

In one embodiment, V is the standard deviation of the magnitudes $|z_j|$ over the window of size k .

5. Collapse and classification:

A collapse operator $F: (\mathbb{C}, \mathbb{R}_{\geq 0}, [0, 1], \Sigma) \rightarrow \{\text{TRUE}, \text{FALSE}, \text{NEUTRAL}, \text{UNDECIDABLE}\} \times \Lambda$

takes as input the current state z_t , the volatility v , the contradiction density c , and the domain label d , and outputs a ternary truth value together with a classification label λ in a finite set Λ of epistemic categories:

$$(\tau, \lambda) = F(z_t, v, c, d)$$

In one embodiment, $\tau \in \{\text{TRUE}, \text{FALSE}, \text{NEUTRAL}, \text{UNDECIDABLE}\}$ and $\lambda \in \{\text{ontological_error}, \text{paradox}, \text{epistemic_gap}, \text{structural_noise}\}$.

6. Overall system mapping:

The overall system behaves as a mapping M from input queries to ternary truth values with epistemic classification:

$$M: Q \rightarrow \{\text{TRUE}, \text{FALSE}, \text{NEUTRAL}, \text{UNDECIDABLE}\} \times \Lambda$$

defined by the composition:

$$M(q) = F(z_t, v, c, d)$$

where:

$$c = C(q)$$

$$e = E(q)$$

$$d = D(q)$$

$$z_0 = S(c, e)$$

$$z_t = K^t(z_0, c) \text{ (iterated application of } K)$$

$$v = V(z_{t-k+1}, \dots, z_t)$$

The ternary oscillating logic kernel thus implements a formal pipeline:

$$q \rightarrow (c, e, d) \rightarrow z_0 \rightarrow \{z_t\} \rightarrow v \rightarrow (\tau, \lambda)$$

where the system sustains undecidability when volatility fails to converge, and only permits collapse to TRUE, FALSE, or UNDECIDABLE when the volatility criteria are satisfied.

[0013] CLAIMS:

1. A method for ternary oscillating logic in a computing system, comprising: initializing a complex-valued state vector; transforming an input query into a quantifiable contradiction density signal and evidence signal via semantic analysis; mapping said signals to a complex-valued state vector stored in memory; iteratively applying gain, loss, entropy, and decay transformations to produce oscillatory behavior; measuring volatility of the oscillation; determining collapse to TRUE, FALSE, or UNDECIDABLE when volatility falls below a

threshold; classifying persistent non-collapse states; wherein the method sustains undecidability and prevents premature binary resolution.

2. The method of claim 1, wherein contradiction density is derived from linguistic markers, and the evidence score adjusts oscillation parameters.

3. The method of claim 1, further comprising domain extraction and epistemic gap detection for unfamiliar domains.

4. The method of claim 1, wherein volatility is computed as a standard deviation of magnitudes in a fixed-length history window.

5. The method of claim 1, wherein non-collapse classification prioritizes `ontological_error` when the evidence score is zero, and axioms are absent.

6. A system comprising a processor and memory storing instructions that, when executed, perform the method of claim 1.

7. The system of claim 6, wherein undecidability triggers domain escalation or specialist module deployment.

8. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to perform the method of claim 1.

9. The method of claim 1, wherein the epistemic classification includes at least: `ontological_error`, `paradox`, `epistemic_gap`, and `structural_noise`, and wherein each classification triggers a distinct downstream action, and wherein said first and second threshold values are configurable based on domain context.

10. The method of claim 9, wherein said domain context includes at least: medical diagnosis, scientific computing, and legal analysis.

11. A method for reducing AI hallucination in high-stakes computing applications, comprising: receiving a query; computing contradiction density and evidence score; executing the ternary oscillating logic kernel of claim 1; upon detecting UNDECIDABLE status with `epistemic_gap` classification, deploying a specialist reasoning module for the identified domain; logging the UNDECIDABLE state and specialist deployment in a tamper-evident audit trail; wherein the method reduces hallucination rates by preventing output generation when evidence is insufficient.

12. The method of claim 1, wherein determining a non-collapse state triggers an interrupt signal that inhibits subsequent token generation layers, thereby preventing power consumption on invalid inference chains.

13. A method for early-stop detection in combinatorial search problems, comprising: executing a search algorithm that generates candidate solutions; concurrently executing the ternary oscillating logic kernel of claim 1 to monitor contradiction density; upon detecting sustained oscillation, halting the search algorithm and escalating to a deterministic solver; wherein the method reduces computational waste by detecting combinatorial explosion before exhausting the search space.